DP Computer Science: A3.2.6 Database Scenarios

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define **First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF)**
- Identify violations of **1NF, 2NF, and 3NF** in a given database table
- **Decompose** an unnormalised database table into a set of tables in **3NF**
- Explain the **purpose and benefits** of database normalisation
- Apply the concepts of **primary keys, foreign keys, and functional dependencies** in the normalisation process

---

Key Vocabulary

Term	Definition
1NF	First Normal Form: each cell contains a single atomic value; no repeating groups
2NF	Second Normal Form: in 1NF and no partial dependencies (non-key attributes depend on the entire composite key)
3NF	Third Normal Form: in 2NF and no transitive dependencies (non-key attributes depend only on the primary key)
Decomposition	The process of splitting a table into multiple tables to achieve normalisation
Functional dependency	A relationship where one attribute determines another (e.g., `StudentID → Name`)

Term	Definition
Partial dependency	A non-key attribute depends on only part of a composite primary key
Transitive dependency	A non-key attribute depends on another non-key attribute
Primary key	A unique identifier for each record in a table
Foreign key	A field that references the primary key of another table

---

IB ATL Elements

Skill	Description
Thinking	Critical thinking (analysing table structures to identify redundancies and dependencies); problem-solving (devising strategies to decompose tables)
Communication	Presenting (explaining the rationale behind each normalisation step); collaborating (working with peers)
Self-Management	Organisation (structuring understanding into a logical sequence); time management

---

2. Introduction to Normalisation

What is Database Normalisation?

> **Database normalisation is the process of organising data to reduce redundancy, improve data integrity, and eliminate dependency issues.**

The Problem Normalisation Solves

```text
┌─────────────────────────────────────────────────────────────────┐
│              WHY NORMALISE?                                   │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              DATA REDUNDANCY                          │  │
│    │                                                         │  │
│    │    Same data stored in multiple places                │  │
│    │    • Wastes storage space                              │  │
│    │    • Update anomalies (must update in multiple places)│  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              DATA INTEGRITY                           │  │
│    │                                                         │  │
│    │    Inconsistent or incomplete data                     │  │
│    │    • Insertion anomalies (can't add data without      │  │
│    │      other data)                                       │  │
│    │    • Deletion anomalies (lose data when deleting)     │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              DEPENDENCY ISSUES                        │  │
│    │                                                         │  │
│    │    Incorrect dependencies lead to data inconsistency   │  │
│    │    • Partial dependencies (2NF)                       │  │
│    │    • Transitive dependencies (3NF)                    │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

The Normal Forms Checklist

Question	Normal Form
Are all values atomic (single) and no repeating groups?	**1NF**
Are all non-key attributes fully dependent on the **entire** primary key?	**2NF**

Question	Normal Form
Are all non-key attributes dependent **only** on the primary key (no transitive dependencies)?	**3NF**

3. Library Management System Example

Unnormalised Table

```text
┌──────────────────────────────────────────────────────────────────┐
│              UNNORMALISED TABLE                                │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │                LIBRARY_LOANS                           │  │
│    ├──────────────────────────────────────────────────────────┤  │
│    │  LoanID │ BorrowerName │ BorrowerAddr │ BookTitle │   │  │
│    │         │              │              │ Author    │   │  │
│    ├─────────┼──────────────┼──────────────┼───────────┼───┤  │
│    │  1      │ Alice Smith  │ 1 Main St    │ SQL Guide │   │  │
│    │         │              │              │ John Doe  │   │  │
│    │  1      │ Alice Smith  │ 1 Main St    │ Python    │   │  │
│    │         │              │              │ Jane Roe  │   │  │
│    │  2      │ Bob Jones    │ 2 Oak Ave    │ Data      │   │  │
│    │         │              │              │ Structures│   │  │
│    │         │              │              │ Sam Lee   │   │  │
│    └─────────┴──────────────┴──────────────┴───────────┴───┘  │
│                                                                  │
│    ❌ Multiple books per loan (repeating groups)               │
│    ❌ Redundant borrower information                           │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Step 1: Normalise to 1NF

Requirements:

- Atomic values (one value per cell)
- No repeating groups
- Define a primary key (composite key)

```text
┌─────────────────────────────────────────────────────────────────────┐
│              AFTER 1NF                                             │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│    ┌───────────────────────────────────────────────────────────┐    │
│    │                LIBRARY_LOANS                            │   │
│    │   Primary Key: (LoanID, BookTitle)                     │   │
│    ├───────────────────────────────────────────────────────────┤   │
│    │  LoanID (PK)│BookTitle (PK)│BorrowerName│BorrowerAddr│   │   │
│    │             │              │            │ Author     │   │   │
│    ├─────────────┼──────────────┼────────────┼────────────┤   │   │
│    │  1          │ SQL Guide    │ Alice Smith│ 1 Main St  │   │   │
│    │             │              │            │ John Doe   │   │   │
│    │  1          │ Python       │ Alice Smith│ 1 Main St  │   │   │
│    │             │              │            │ Jane Roe   │   │   │
│    │  2          │ Data         │ Bob Jones  │ 2 Oak Ave  │   │   │
│    │             │ Structures   │            │ Sam Lee    │   │   │
│    └─────────────┴──────────────┴────────────┴────────────┘   │
│                                                                     │
│    ✅ Atomic values                                                │
│    ✅ No repeating groups                                          │
│    ✅ Composite key: (LoanID, BookTitle)                          │
│                                                                     │
│    ⚠️ Partial Dependencies exist!                                │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

Partial Dependencies Identified:

- `BorrowerName` depends on `LoanID` (not the whole composite key)
- `BorrowerAddress` depends on `LoanID` (not the whole composite key)
- `Author` depends on `BookTitle` (not the whole composite key)

Step 2: Normalise to 2NF

Requirements:

- In 1NF
- No partial dependencies

```text
┌─────────────────────────────────────────────────────────────────┐
│              AFTER 2NF                                         │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                  LOANS                                │  │
│    │           Primary Key: LoanID                           │  │
│    ├─────────────────────────────────────────────────────────┤  │
│    │  LoanID (PK) │ BorrowerName │ BorrowerAddress          │  │
│    ├──────────────┼──────────────┼──────────────────────────┤  │
│    │  1           │ Alice Smith  │ 1 Main St                │  │
│    │  2           │ Bob Jones    │ 2 Oak Ave                │  │
│    └──────────────┴──────────────┴──────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                  BOOKS                                │  │
│    │           Primary Key: BookTitle                        │  │
│    ├─────────────────────────────────────────────────────────┤  │
│    │  BookTitle (PK) │ Author                               │  │
│    ├─────────────────┼───────────────────────────────────────┤  │
│    │  SQL Guide      │ John Doe                            │  │
│    │  Python         │ Jane Roe                            │  │
│    │  Data Structures│ Sam Lee                             │  │
│    └─────────────────┴───────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │                LOAN_ITEMS (Junction)                  │  │
│    │   Primary Key: (LoanID, BookTitle)                    │  │
│    ├─────────────────────────────────────────────────────────┤  │
│    │  LoanID (PK)(FK) │ BookTitle (PK)(FK)                 │  │
│    ├──────────────────┼──────────────────────────────────────┤  │
│    │  1               │ SQL Guide                           │  │
│    │  1               │ Python                              │  │
│    │  2               │ Data Structures                    │  │
│    └──────────────────┴──────────────────────────────────────┘  │
```

```
│                                                    │
│   ✅ No partial dependencies                       │
│                                                    │
│   ⚠️ Transitive Dependency exists!                 │
│                                                    │
└────────────────────────────────────────────────────┘
```

Transitive Dependency Identified:

- In the Loans table, `BorrowerAddress` depends on `BorrowerName` (a non-key attribute)
- The dependency chain is: `LoanID → BorrowerName → BorrowerAddress`

Step 3: Normalise to 3NF

Requirements:

- In 2NF
- No transitive dependencies

```text
┌────────────────────────────────────────────────────┐
│              AFTER 3NF                             │
├────────────────────────────────────────────────────┤
│                                                    │
│    ┌──────────────────────────────────────────────┐ │
│    │                 LOANS                        │ │
│    │           Primary Key: LoanID                │ │
│    ├──────────────────────────────────────────────┤ │
│    │  LoanID (PK) │ BorrowerID (FK)               │ │
│    ├──────────────┼───────────────────────────────┤ │
│    │  1           │ 101                           │ │
│    │  2           │ 102                           │ │
│    └──────────────┴───────────────────────────────┘ │
│                                                    │
│    ┌──────────────────────────────────────────────┐ │
│    │               BORROWERS                      │ │
│    │           Primary Key: BorrowerID            │ │
│    ├──────────────────────────────────────────────┤ │
│    │  BorrowerID (PK) │ BorrowerName │ BorrowerAddress │ │
│    ├──────────────────┼──────────────┼─────────────────┤ │
```

```
│    │  101              │ Alice Smith  │ 1 Main St          │  │
│    │  102              │ Bob Jones    │ 2 Oak Ave          │  │
│    └───────────────────┴──────────────┴────────────────────┘  │
│                                                               │
│    ┌──────────────────────────────────────────────────────┐   │
│    │                  BOOKS                              │   │
│    │           Primary Key: BookTitle                     │  │
│    ├──────────────────────────────────────────────────────┤   │
│    │  BookTitle (PK) │ Author                             │  │
│    ├─────────────────┼────────────────────────────────────┤  │
│    │  SQL Guide      │ John Doe                          │   │
│    │  Python         │ Jane Roe                          │   │
│    │  Data Structures│ Sam Lee                           │   │
│    └─────────────────┴────────────────────────────────────┘  │
│                                                               │
│    ┌──────────────────────────────────────────────────────┐   │
│    │                LOAN_ITEMS                           │   │
│    │   Primary Key: (LoanID, BookTitle)                  │   │
│    ├──────────────────────────────────────────────────────┤   │
│    │  LoanID (PK)(FK) │ BookTitle (PK)(FK)            │   │
│    ├──────────────────┼───────────────────────────────────┤  │
│    │  1               │ SQL Guide                      │   │
│    │  1               │ Python                         │   │
│    │  2               │ Data Structures               │   │
│    └──────────────────┴───────────────────────────────────┘  │
│                                                               │
│    ✅ No transitive dependencies                             │
│    ✅ All non-key attributes depend only on the primary key  │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

---

4. Student Activity: School Management System

Unnormalised Table

```text
┌───────────────────────────────────────────────────────────────┐
│              SCHOOL MANAGEMENT TABLE                         │
```

```
├─────────────────────────────────────────────────────────────┤
│                                                             │
│    ┌──────────────────────────────────────────────────────┐ │
│    │  StudentID │ StudentName │ Subject │ Teacher │ Grade  │ │
│    ├────────────┼─────────────┼─────────┼─────────┼────────┤ │
│    │  S001      │ Alice       │ Math    │ Mr. A   │ A      │ │
│    │  S001      │ Alice       │ English │ Ms. B   │ B      │ │
│    │  S002      │ Bob         │ Math    │ Mr. A   │ A      │ │
│    │  S003      │ Charlie     │ Science │ Mr. C   │ C      │ │
│    │  S003      │ Charlie     │ Math    │ Mr. A   │ B      │ │
│    └────────────┴─────────────┴─────────┴─────────┴────────┘ │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

Task

Normalise this table to **3NF**, showing:

1. The table at each normal form
2. Primary and foreign keys at each stage
3. Explanation of the dependencies being resolved

---

5. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Welcome; learning objectives; normalisation importance
What is Normalisation?	5 min	Definition; goals; three normal forms
1NF	10 min	Library Loans table; transform to 1NF
2NF	10 min	Partial dependencies; normalise to 2NF

Phase	Time	Activity
3NF	10 min	Transitive dependencies; normalise to 3NF
Student Activity	15 min	School Management System normalisation
Wrap-up	5 min	Review; summary; Q&A

---

6. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Student Activity	Evaluate ability to normalise tables to 3NF
Exit Ticket	Gauge understanding and identify areas needing clarification

---

7. Differentiation

Level	Strategy
Support	Provide step-by-step guidance and visual aids
Challenge	Normalise more complex multi-table databases

---

8. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.2.6 with examples)
- **Worksheet:** School Management System normalisation

- **Worksheet Answers**
- **Quizlet** (A3.2.6 vocabulary flashcards)

---

9. Summary: Key Takeaways

Normal Form	Key Rule	Problem Solved
1NF	Atomic values; no repeating groups	Multi-valued attributes
2NF	No partial dependencies	Data duplication; update anomalies
3NF	No transitive dependencies	Insertion, deletion, update anomalies

```text
┌─────────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                              │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ✅ Normalisation reduces redundancy                        │
│    ✅ 1NF: Atomic, no repeating groups                        │
│    ✅ 2NF: Whole key (no partial dependencies)              │
│    ✅ 3NF: Nothing but the key (no transitive dependencies) │
│    ✅ Partial dependency = part of composite key            │
│    ✅ Transitive dependency = non-key → non-key             │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

> "*Normalisation is the process of organising data to reduce redundancy—'the key, the whole key, and nothing but the key.*'"